

Desenvolvimento Web com .NET

Aula 5 – Data Annotations e Seeding

Prof. Me. Lucas Teodoro dos Santos

UNIP

02 de Setembro de 2026

1 Data Annotations

2 Seeding

Data Annotations

Data Annotations são atributos adicionados às classes e propriedades para fornecer informações ao Entity Framework Core.

Elas podem ser utilizadas para definir:

- Chaves primárias e estrangeiras;
- Campos obrigatórios;
- Tamanhos mínimos e máximos;
- Intervalos de valores;
- Nomes de tabelas e colunas;
- Propriedades que não serão armazenadas.

Na abordagem *Code First*, o EF Core analisa as classes da aplicação para construir o modelo do banco de dados.

Por padrão, o EF Core utiliza convenções. Por exemplo:

- `Id` é reconhecido como chave primária;
- `string` é mapeada como uma coluna de texto;
- `int` é mapeado como uma coluna de número inteiro.

As Data Annotations permitem complementar ou alterar essas convenções.

Quando o modelo é construído ou uma migration é criada, o EF Core:

- 1 Analisa as classes das entidades;
- 2 Localiza os atributos presentes nas classes e propriedades;
- 3 Interpreta as configurações informadas;
- 4 Monta o modelo da aplicação;
- 5 Utiliza esse modelo para gerar as migrations.

Importante

A Data Annotation apenas fornece uma configuração. Quem interpreta e aplica essa configuração é o Entity Framework Core.

Como utilizar

Primeiramente, importamos os namespaces:

```
1 using System.ComponentModel.DataAnnotations;  
2 using System.ComponentModel.DataAnnotations.Schema;
```

Depois, adicionamos os atributos sobre a classe ou propriedade:

```
1 public class Medico  
2 {  
3     [Key]  
4     public int Id { get; set; }  
5  
6     [Required]  
7     [MaxLength(100)]  
8     public string Nome { get; set; } = string.Empty;  
9 }
```

O atributo [Key] define a chave primária da entidade.

```
1 public class Medico
2 {
3     [Key]
4     public int Id { get; set; }
5
6     public string Nome { get; set; } = string.Empty;
7 }
```

Observação

O EF Core já reconhece propriedades chamadas Id ou MedicoId como chaves primárias por convenção. O atributo permite deixar essa configuração explícita.

Required

O atributo [Required] indica que uma propriedade é obrigatória.

```
1 [Required]  
2 public string Nome { get; set; } = string.Empty;
```

No banco de dados, essa configuração pode resultar em uma coluna que não aceita valores nulos:

Nome VARCHAR(...) NOT NULL

Observação

Tipos não anuláveis do C#, como `string`, também são interpretados pelo EF Core como obrigatórios.

MaxLength e MinLength

[MaxLength] define o tamanho máximo e [MinLength] define o tamanho mínimo aceito.

```
1 [Required]
2 [MinLength(3)]
3 [MaxLength(100)]
4 public string Nome { get; set; } = string.Empty;
```

- MaxLength(100) pode gerar uma coluna com tamanho 100;
- MinLength(3) é utilizado principalmente na validação;
- O tamanho mínimo não gera automaticamente uma restrição no banco.

StringLength

O atributo [StringLength] permite definir os tamanhos máximo e mínimo de uma propriedade textual.

```
1 [Required]
2 [StringLength(80, MinimumLength = 3)]
3 public string Especialidade { get; set; }
4 = string.Empty;
```

- O primeiro valor representa o tamanho máximo;
- MinimumLength representa o tamanho mínimo;
- O tamanho máximo pode influenciar a coluna criada pelo EF Core;
- O tamanho mínimo atua principalmente na validação.

O atributo [Range] define um intervalo de valores aceitos por uma propriedade numérica.

```
1 [Range(0, 10000)]  
2 public int QuantidadeEstoque { get; set; }  
3  
4 [Range(0.01, 999999.99)]  
5 public decimal Preco { get; set; }
```

Atenção

[Range] é utilizado principalmente na validação da aplicação. Ele não cria automaticamente uma restrição CHECK no banco de dados.

Table

O atributo [Table] define o nome da tabela utilizada para armazenar a entidade.

```
1 [Table("Medicos")]
2 public class Medico
3 {
4     public int Id { get; set; }
5     public string Nome { get; set; } = string.Empty;
6 }
```

Sem essa configuração, o nome da tabela será definido pelas convenções e pelo nome utilizado no DbSet.

O atributo [Column] permite definir o nome da coluna:

```
1 [Column("CRM")]  
2 public string Crm { get; set; } = string.Empty;
```

Também permite definir o tipo utilizado no banco:

```
1 [Column(TypeName = "numeric(10,2)")]  
2 public decimal Preco { get; set; }
```

No segundo exemplo, a coluna poderá armazenar dez dígitos, sendo dois após a vírgula.

O atributo [ForeignKey] indica qual propriedade representa uma chave estrangeira.

```
1 public class Medico
2 {
3     public int Id { get; set; }
4     public string Nome { get; set; } = string.Empty;
5     e
6     [ForeignKey(nameof(Especialidade))]
7     public int EspecialidadeId { get; set; }
8
9     public Especialidade Especialidade { get; set; }
10    = null!;
11 }
```

NotMapped

O atributo [NotMapped] informa que uma propriedade não deverá ser armazenada no banco de dados.

```
1 public class Produto
2 {
3     public int Id { get; set; }
4     public decimal Preco { get; set; }
5     public int Quantidade { get; set; }
6
7     [NotMapped]
8     public decimal ValorTotal
9     {
10         get { return Preco * Quantidade; }
11     }
12 }
```


Resumo das principais anotações

Atributo	Utilização
[Key]	Define a chave primária.
[Required]	Define uma propriedade obrigatória.
[MinLength]	Define o tamanho mínimo aceito.
[MaxLength]	Define o tamanho máximo aceito.
[StringLength]	Define os tamanhos mínimo e máximo de uma string.
[Range]	Define um intervalo de valores aceitos.
[Table]	Define o nome da tabela.
[Column]	Define o nome ou o tipo da coluna.
[ForeignKey]	Indica uma propriedade de chave estrangeira.
[NotMapped]	Impede o mapeamento de uma propriedade.

As Data Annotations modificam o modelo utilizado pelo EF Core, mas não alteram imediatamente o banco de dados.

Após adicionar ou modificar uma anotação, devemos:

- ❶ Criar uma nova migration;
- ❷ Conferir as alterações geradas;
- ❸ Aplicar a migration ao banco de dados.

Criando e Aplicando a Migration

Visual Studio Community – Console do Gerenciador de Pacotes

```
1 Add-Migration AdicionarDataAnnotations
2 Update-Database
```

VS Code – Terminal

```
1 dotnet ef migrations add AdicionarDataAnnotations
2 dotnet ef database update
```

Seeding

O que é Seeding?

Seeding é o processo de inserir dados iniciais no banco de dados de uma aplicação.

Esses dados podem ser utilizados para:

- Disponibilizar registros de exemplo para testes e desenvolvimento;
- Criar dados básicos necessários para o funcionamento do sistema;
- Evitar que uma tabela seja iniciada totalmente vazia;
- Facilitar a demonstração das telas da aplicação.

No projeto de agendamento, médicos iniciais podem ser inseridos para que sejam apresentados na tela de listagem quando a aplicação for executada.

Formas de realizar o Seeding

Existem diferentes formas de inserir dados iniciais em uma aplicação com EF Core.

- **Com HasData():** os dados são definidos no DbContext e inseridos por meio de uma migration;
- **Com uma classe de serviço:** uma classe, como SeedingService, é responsável por verificar e inserir os dados;
- **Manualmente:** os registros podem ser incluídos diretamente no banco de dados ou pela própria aplicação.

Nesta aula, será utilizada uma classe de serviço para organizar a lógica de população inicial dos médicos.

Classe SeedingService

A classe SeedingService será criada dentro da pasta Data do projeto.

Ela será responsável por concentrar a lógica de inserção dos dados iniciais no banco de dados.

Como essa classe precisa acessar o ApplicationDbContext, sua localização na pasta Data ajuda a organizar os elementos relacionados ao acesso e à preparação dos dados da aplicação.

```
1 namespace Agendamento.Data
2 {
3     public class SeedingService
4     {
5
6     }
7 }
```

Classe SeedingService - Criando o objeto de contexto

Para realizar operações no banco de dados, como consultar, inserir, alterar ou remover registros, a classe SeedingService precisa de um objeto do tipo AppDbContext.

Esse objeto será armazenado em uma propriedade privada da classe:

```
1 private readonly AppDbContext _context;
```

Por meio do contexto, a aplicação pode executar operações como:

- Consultar registros existentes (Find(), ToList());
- Adicionar novos registros (Add(), AddRange());
- Alterar registros (Update());
- Remover registros (Remove());
- Salvar as alterações no banco de dados (SaveChanges()).

A palavra `readonly` indica que a variável poderá receber um valor apenas durante sua declaração ou no construtor da classe. Após isso, o objeto armazenado em `_context` não poderá ser substituído por outro.

Classe SeedingService - Recebendo o contexto por DI

De forma convencional, o contexto poderia ser criado manualmente com `new`:

```
1 AppDbContext context = new AppDbContext(...);
```

Nesse caso, seria necessário criar também as configurações utilizadas pelo contexto, como o provedor e a conexão com o banco de dados.

Com a **Injeção de Dependência (DI)**, o contexto é solicitado como parâmetro no construtor da classe:

```
1 public SeedingService(AppDbContext context)
2 {
3     _context = context;
4 }
```

O framework será responsável por criar e fornecer à classe um objeto `AppDbContext` já configurado.

Classe SeedingService - DI no SeedingService

Quando for necessário criar um objeto SeedingService, o framework analisará o construtor da classe:

```
1 public SeedingService(AppDbContext context)
```

Como o construtor solicita um AppDbContext, o framework:

- Localiza o registro do AppDbContext;
- Cria ou recupera um objeto de contexto já configurado;
- Envia esse objeto pelo parâmetro context;
- Armazena a referência na variável _context;
- Retorna o objeto SeedingService pronto para uso.

Dessa forma, a classe recebe a dependência necessária sem precisar criar ou configurar manualmente a conexão com o banco de dados.

Classe SeedingService - Método Popula()

O método Popula() será responsável por verificar se existem médicos cadastrados e inserir os dados iniciais quando necessário.

Inicialmente, é criada a estrutura do método:

```
1 public void Popula()  
2 {  
3  
4 }
```

Como o método não precisa retornar nenhuma informação, é utilizado o tipo void.

Classe SeedingService - Verificando registros existentes

Antes de inserir os médicos, é necessário verificar se a tabela já possui registros.

```
1      if (_context.Medicos.Any())  
2      {  
3          return;  
4      }
```

- `_context.Medicos` representa o conjunto de médicos acessado pelo contexto;
- `Any()` verifica se existe pelo menos um registro;
- Caso existam médicos cadastrados, `return` encerra o método sem inserir novos dados.

Assim, os dados iniciais não serão inseridos novamente quando a população for executada mais de uma vez.

Classe SeedingService - Criando os médicos iniciais

Caso não existam registros na tabela, objetos do tipo Medico são criados com os dados iniciais da aplicação.

```
1      Medico medico1 = new Medico
2      {
3          Id = 1,
4          Nome = "Ana Souza",
5          Crm = "12345",
6          Especialidade = "Cardiologia"
7      };
8
9      Medico medico2 = new Medico
10     {
11         Id = 2,
12         Nome = "Carlos Lima",
13         Crm = "67890",
14         Especialidade = "Ortopedia"
15     };
```

Cada objeto representa um médico que será inserido posteriormente no banco de dados.

Classe SeedingService - Adicionando e salvando os registros

Após criar os objetos, os médicos são adicionados ao contexto:

```
1 _context.Medicos.AddRange(medico1, medico2);
```

O método `AddRange()` permite adicionar vários objetos de uma vez. Nesse momento, os objetos são preparados pelo contexto, mas ainda não foram gravados no banco de dados.

Para efetivar a inserção, é utilizado:

```
1 _context.SaveChanges();
```

O método `SaveChanges()` envia as alterações realizadas pelo contexto para o banco de dados.

Registrando o SeedingService na Program.cs (1/3)

Para que o framework consiga criar e fornecer um objeto `SeedingService` com suas dependências, a classe deve ser registrada no `Program.cs`.

O registro é realizado antes da criação da aplicação:

```
builder.Services.AddScoped<SeedingService>();
```

O método `AddScoped()` informa que um objeto `SeedingService` será criado dentro de cada escopo de utilização.

Esse ciclo de vida é adequado porque o `SeedingService` depende do `AppDbContext`, que também é utilizado dentro de um escopo.

Entendendo o ciclo de vida dos serviços (2/3)

Ao registrar um serviço, é definido por quanto tempo o objeto criado pelo framework ficará disponível na aplicação.

Os principais ciclos de vida são:

- **Singleton:** uma única instância é criada e utilizada durante toda a execução da aplicação;
- **Scoped:** uma instância é criada para cada requisição HTTP ou escopo de utilização;
- **Transient:** uma nova instância é criada sempre que o serviço é solicitado.

No `Program.cs`, cada ciclo de vida possui um método de registro:

```
1 builder.Services.AddSingleton<MeuServico>();  
2 builder.Services.AddScoped<MeuServico>();  
3 builder.Services.AddTransient<MeuServico>();
```


Executando o SeedingService (1/2)

O código responsável pelo seeding deve ser adicionado no Program.cs, dentro da verificação de ambiente da aplicação.

```
1 // Verifica se a aplicacao nao esta em desenvolvimento
2 if (!app.Environment.IsDevelopment())
3 {
4     app.UseExceptionHandler("/Home/Error");
5     app.UseHsts();
6 }
7 else
8 {
9     // Executado apenas no ambiente de desenvolvimento
10    using (var scope = app.Services.CreateScope())
11    {
12        var seedingService = scope.ServiceProvider
13            .GetRequiredService<SeedingService>();
14
15        seedingService.Popula();
16    }
17 }
```

Executando o SeedingService (2/2)

Esse bloco é responsável por criar um escopo temporário, obter um objeto SeedingService e executar o método Popula().

```
1 using (var scope = app.Services.CreateScope())  
2 {  
3     var seedingService = scope.ServiceProvider  
4     .GetRequiredService<SeedingService>();  
5  
6     seedingService.Popula();  
7 }
```

De forma geral, cada parte possui a seguinte responsabilidade:

- CreateScope(): cria um escopo temporário;
- ServiceProvider: acessa os serviços registrados;
- GetRequiredService(): obtém o objeto SeedingService;
- Popula(): executa a inserção dos dados iniciais.

Resultado do Seeding

Após executar a aplicação no ambiente de desenvolvimento, o método `Popula()` será chamado durante a inicialização.

Como a tabela de médicos está vazia, os objetos criados no `SeedingService` serão inseridos e salvos no banco de dados.

Assim, ao consultar a tabela `Medicos` no PostgreSQL, os registros iniciais já deverão estar disponíveis:

- Ana Souza – Cardiologia;
- Carlos Lima – Ortopedia.

Caso a aplicação seja executada novamente, o método `Any()` identificará que já existem médicos cadastrados e evitará uma nova inserção dos mesmos dados.

Nesta aula, foram abordados os seguintes conteúdos:

- Utilização de *Data Annotations* para definir regras de validação e mapeamento das propriedades da entidade;
- Uso de atributos como `Required`, `MaxLength` e `StringLength`;
- Criação e aplicação de uma migration para atualizar a estrutura do banco de dados;
- Utilização de *seeding* para inserir dados iniciais no banco de dados;
- Criação da Classe *SeedingService*;
- Configuração para executar o *seeding* na inicialização da aplicação;

Obrigado!

Dúvidas?